

Đề tài: Thiết kế & Cài đặt Trò chơi Minesweeper với AI Solver

Nhóm 3

Nguyễn Đình Bằng - 2210298

Trần Minh Quân - 2212822

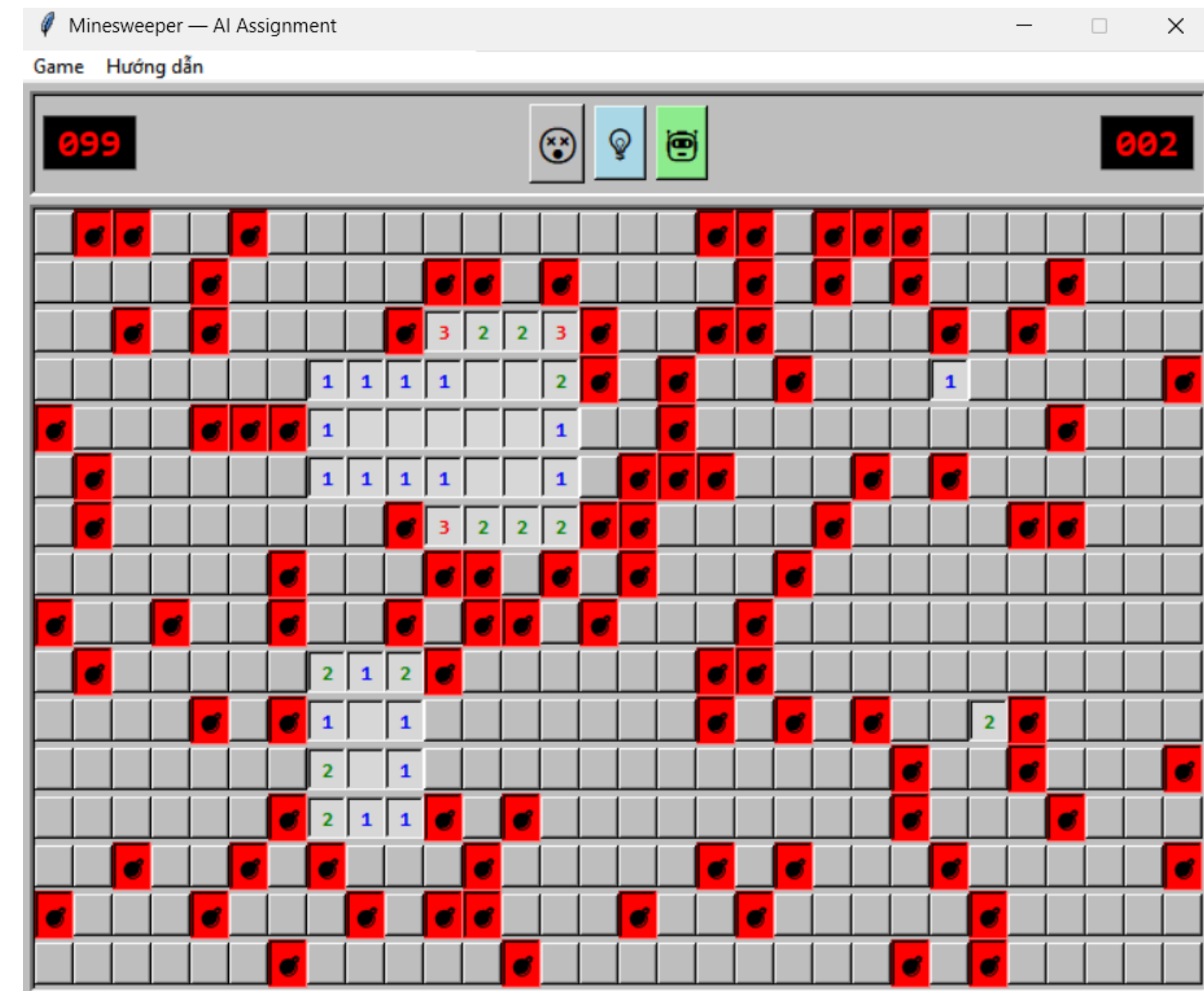
Phạm Lưu Bang - 2410199

Lê Quang Trưng - 2213730

Trần Hữu Luân - 2211954

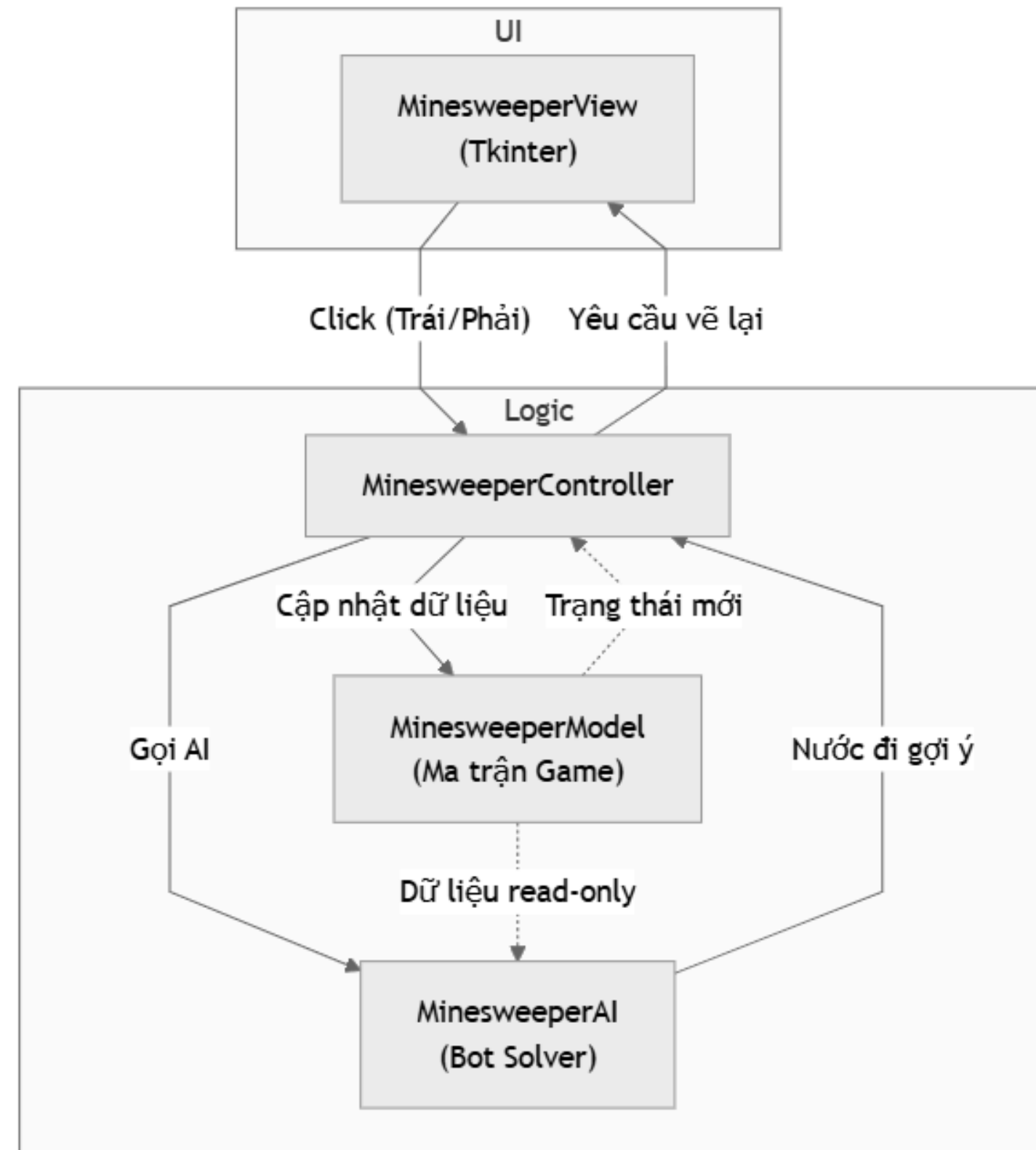
Giới thiệu & Động lực (Introduction & Motivations)

- Minesweeper (Dò mìn) là trò chơi giải đố kinh điển, yêu cầu mở toàn bộ ô an toàn dựa trên manh mối các ô số.
- Động lực thực hiện:
 - Thử nghiệm khả năng của các thuật toán tìm kiếm (Search) và thỏa mãn các ràng buộc.
 - Mô phỏng và vượt qua khả năng suy luận logic của con người.
- Mục tiêu: Tự xây dựng Game từ đầu (không dùng thư viện có sẵn) + Tích hợp bot AI có khả năng tự động giải bảng mìn.



Tổng quan Kiến trúc Hệ thống (System Architecture)

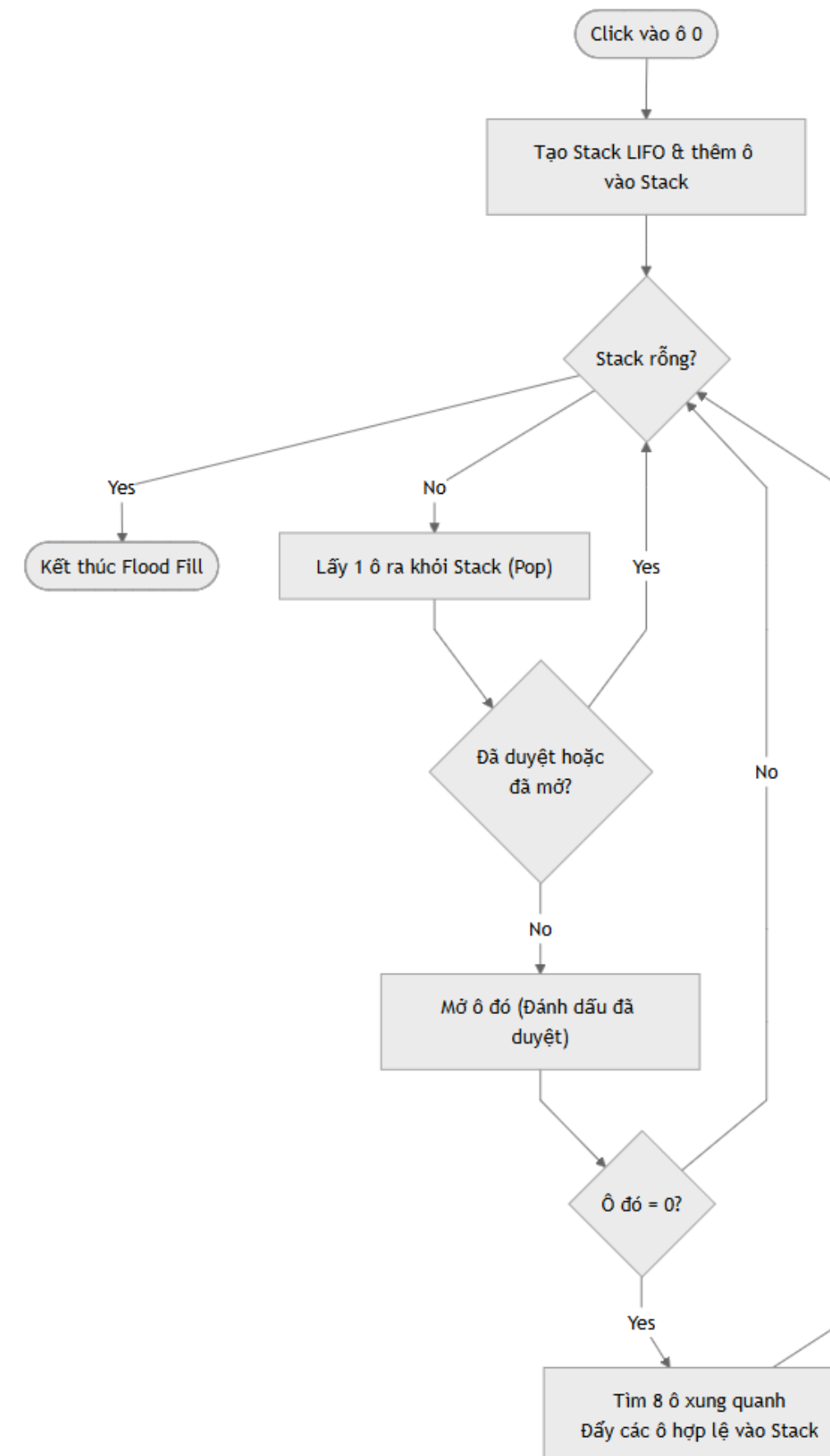
- Mô hình thiết kế: Sử dụng chuẩn Model-View-Controller (MVC).
 - Model: Quản lý ma trận mìn (Board) và trạng thái hiển thị (State). Xử lý thuật toán lỗi.
 - View: Giao diện đồ họa Tkinter (Menu, Grid, Timer, Smiley Button).
 - Controller: Xử lý click chuột, điều phối Model cập nhật và View vẽ lại.
 - AI Solver: Module độc lập, đọc dữ liệu từ Model và tư vấn nước đi.



Thuật toán Cốt lõi của Game (Core Algorithm)

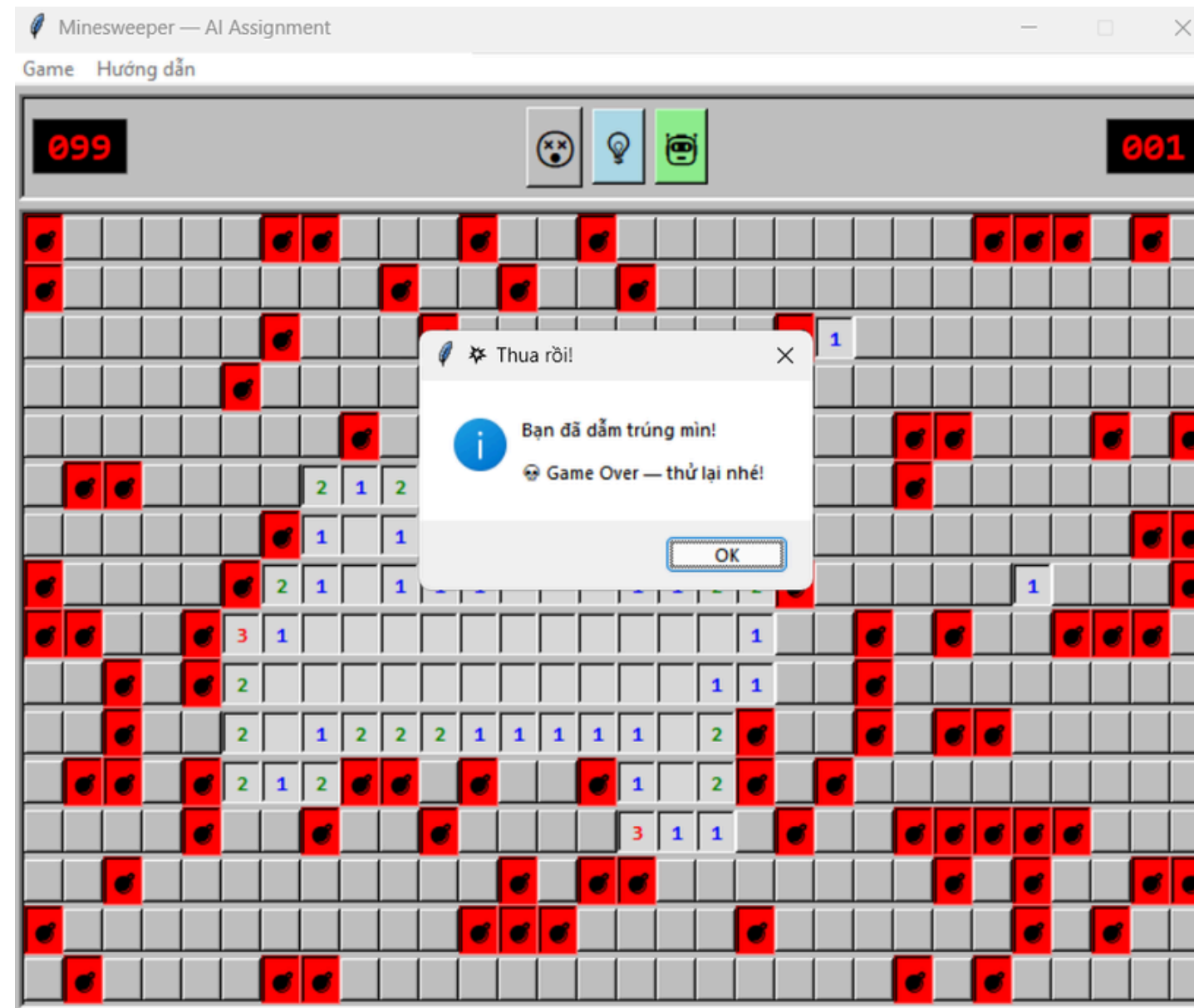
Thuật toán DFS (Depth-First Search) cho cơ chế "Flood Fill":

- Vấn đề: Khi click vào ô 0 (trống), cần mở rộng tất cả ô an toàn liền kề ngay lập tức.
- Giải pháp: Áp dụng Iterative DFS (Duyệt theo chiều sâu không đệ quy).
- Tại sao không dùng đệ quy?: Python giới hạn đệ quy ở khoảng 1000 frames. Trên bảng mìn lớn (30*30), đệ quy sẽ gây lỗi RecursionError làm crash game.
- Cách làm: Dùng cấu trúc dữ liệu Ngăn xếp (Stack - LIFO) tự quản lý, duyệt liên tục 8 hướng đến khi cạn stack. An toàn, tốc độ nhanh.



Tại sao cần AI Solver? (Why AI?)

- Thực trạng: Con người chơi thường mất thời gian suy nghĩ, dễ sai sót hoặc bỏ cuộc khi gặp thế cờ lớn (99 mìn).
- Phân tích bài toán NP-Complete: Giải Minesweeper trong trường hợp tổng quát là bài toán NP-Đầy đủ. Không thể giải bằng một thuật toán đơn giản duy nhất.
- Đề xuất của nhóm: Xây dựng một Hệ thống AI 3 Tầng (3-Tier AI Solver) kết hợp giữa Logic cứng nhắc và Dự đoán mềm dẻo.



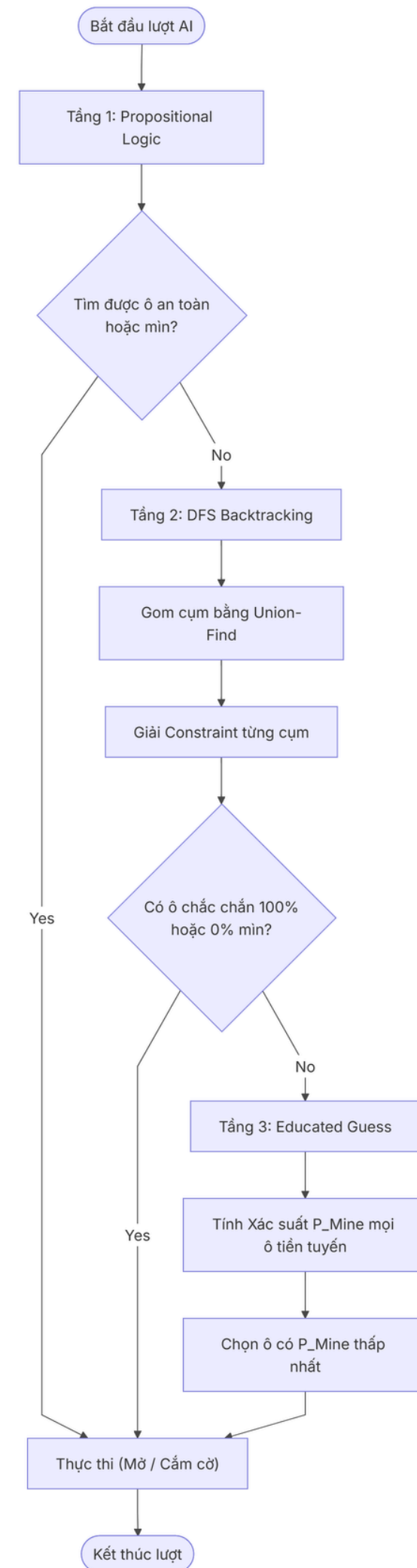
AI Tầng 1 - Propositional Logic (Suy luận Mệnh đề)

- Nguyên lý: Quét toàn bảng, dựa trên số mìn V, số cờ xung quanh F và số ô chưa mở H.
- Quy tắc 1 (All Safe): Nếu $V = F \Rightarrow$ Đã đủ mìn, mở (Reveal) tất cả ô H còn lại.
- Quy tắc 2 (All Mines): Nếu $V - F = H \Rightarrow$ Thiếu đúng H quả mìn, cắm cờ (Flag) tất cả ô H đó.
- Đánh giá: Chạy vòng lặp liên tục ($O(N)$). Giải quyết được 70% - 80% diễn biến của ván game.



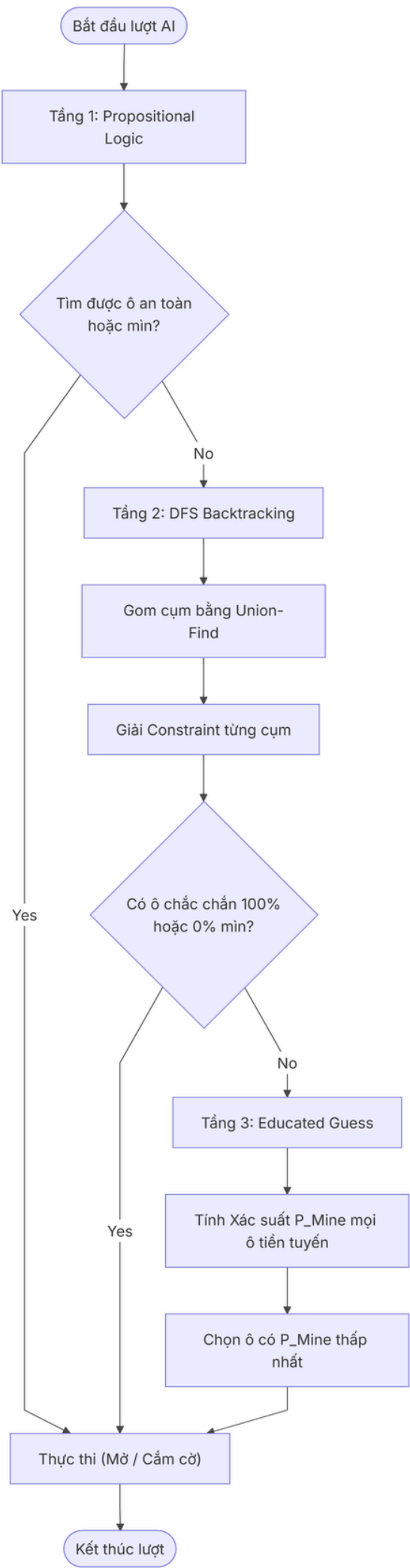
AI Tầng 2 - DFS Backtracking & Constraint Satisfaction

- Kích hoạt khi Tầng 1 gặp ngõ cụt (không có ô nào thỏa mãn All Safe / All Mines).
- Kỹ thuật sử dụng:
 - Union-Find (Gom cụm): Phân chia các ô "tiền tuyến" chưa mở thành các cụm nhỏ độc lập để giảm không gian tìm kiếm.
 - Backtracking & Pruning: Duyệt đệ quy thử các cấu hình (Mìn/AnToàn) cho mỗi cụm ($K \leq 15$). Tỉa ngay các nhánh vi phạm phương trình ô số.
- Kết luận từ Tầng 2: Nếu trong TẤT CẢ các cấu hình hợp lệ, một ô LUÔN LUÔN là Mìn. Tương tự cho ô An toàn.



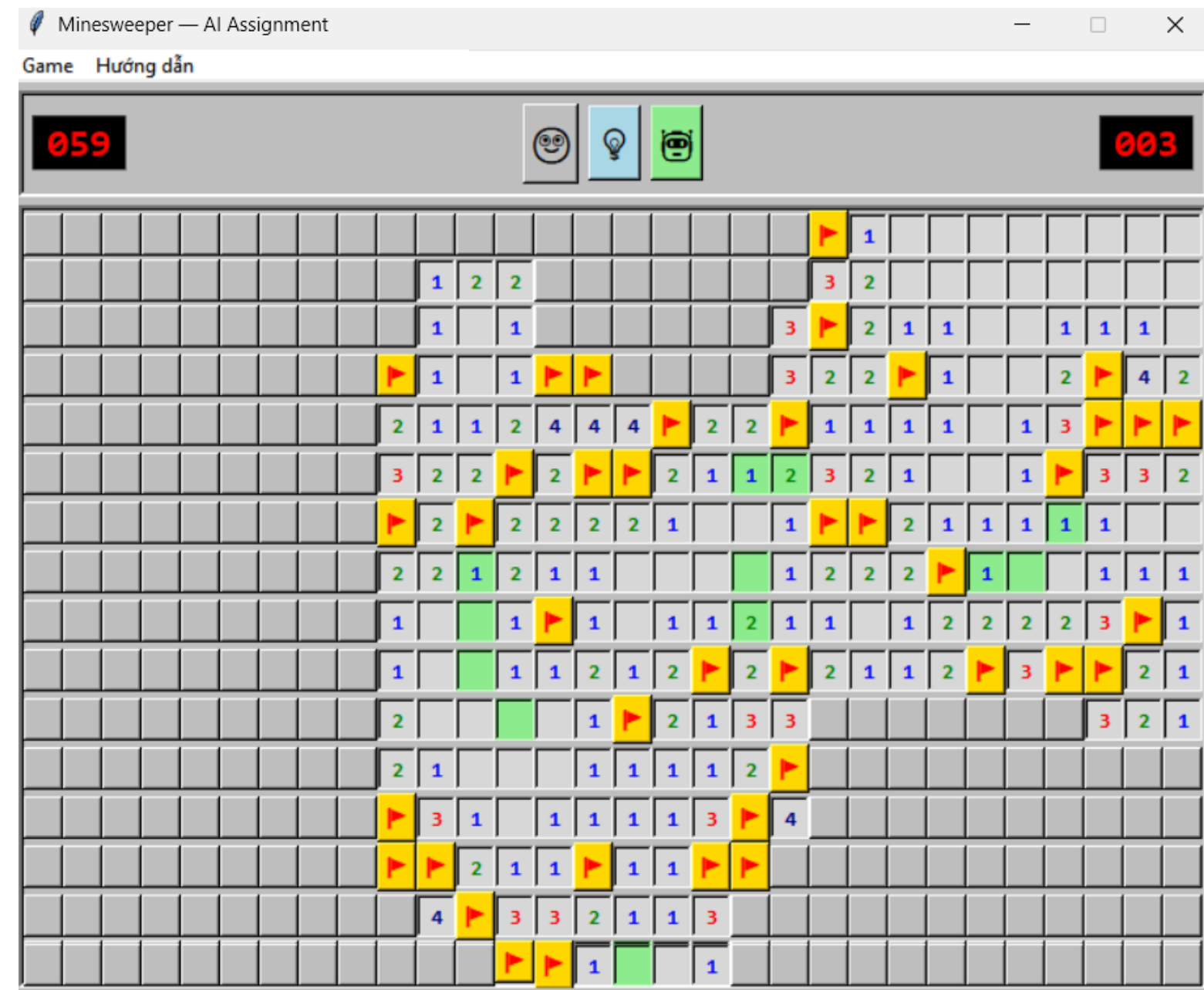
AI Tầng 3 - Educated Guess (Dự đoán Xác suất)

- Kích hoạt khi thế cờ bắt buộc phải đoán (ví dụ: góc 50/50) hoặc Tầng 2 không tìm ra ô nào chắc chắn 100%.
- Thuật toán:
 - Tính xác suất $P(\text{Mine})$ cục bộ cho từng ô quanh các ô số ở tiền tuyến.
 - So sánh với xác suất $P(\text{Mìn}) = \text{Mìn còn lại} / \text{Ô trống còn lại}$.
- Quyết định: Chọn hy sinh click vào ô có xác suất trúng mìn thấp nhất.
- Đánh giá: Tối ưu hóa tối đa khả năng sống sót so với việc người chơi nhắm mắt click bừa.



Trải nghiệm Giao diện & Tính năng Nổi bật (Demo Prep)

- Giao diện: Tkinter phân cấp rõ ràng.
- Tính năng nổi bật:
 - First-Click Safety: Lượt click đầu tiên luôn an toàn (rải mìn trể). Tránh việc thua ngay giây đầu.
 - Hint Mode (): AI đi từng bước một.
 - Auto-Solve Mode (): AI tự động giải liên tục với hiệu ứng chuyển động mượt mà.
 - Chord Reveal: Double-click hỗ trợ mở nhanh.



Slide 10: Kết quả & Phân tích (Results)

Bảng Thống kê Tỷ lệ thắng (Win Rate) của AI (Test tự động 100 ván):

Độ khó	Kích thước	Mật độ mìn	Tỷ lệ thắng ước tính (%)
Dễ	5 × 5	Lên tới 12.00% (3 mìn)	≈ 96.00%
Trung bình	9 × 9	Lên tới 12.35% (10 mìn)	≈ 95.00%
Khó	16 × 16	Lên tới 15.62% (40 mìn)	≈ 78.00%
Chuyên gia	16 × 30	Lên tới 20.62% (99 mìn)	≈ 42.00%

Bảng 1: Thống kê tỷ lệ chiến thắng của AI Solver trên các mức độ khó phổ biến.

Nhận xét: Tỷ lệ thắng cực kỳ xuất sắc ở bảng nhỏ. Ở bảng lớn (99 mìn), tỷ lệ giảm do mật độ mìn quá đặc, ép AI phải nhảy sang Tầng 3 (đoán 50/50) nhiều lần hơn.

Slide 10: Kết quả & Phân tích (Results)

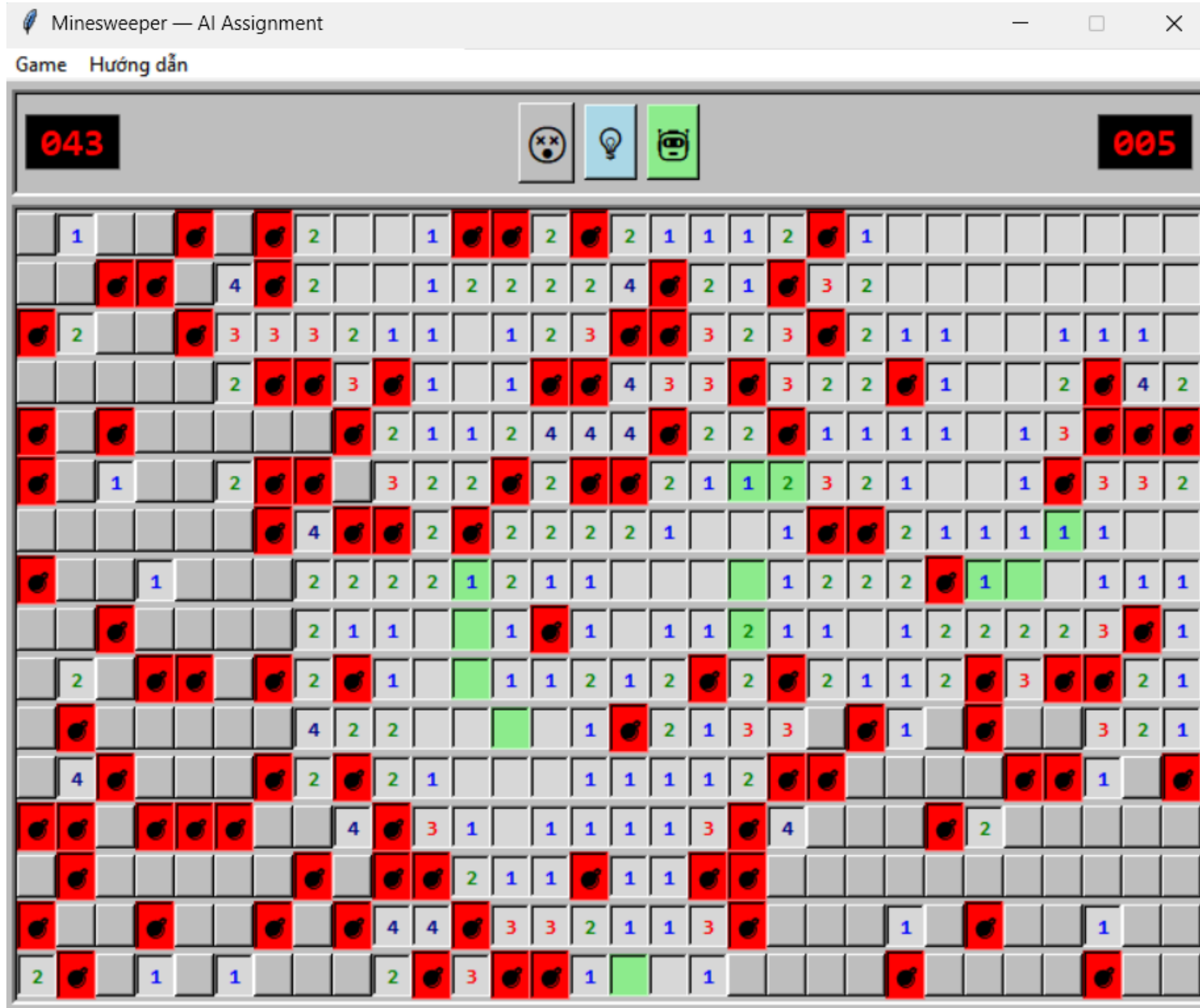
Bảng Thống kê Tỷ lệ thắng (Win Rate) của AI (Test tự động 100 ván):

Độ khó	Kích thước	Mật độ mìn	Tỷ lệ thắng ước tính (%)
Dễ	5×5	Lên tới 12.00% (3 mìn)	$\approx 96.00\%$
Trung bình	9×9	Lên tới 12.35% (10 mìn)	$\approx 95.00\%$
Khó	16×16	Lên tới 15.62% (40 mìn)	$\approx 78.00\%$
Chuyên gia	16×30	Lên tới 20.62% (99 mìn)	$\approx 42.00\%$

Bảng 1: Thống kê tỷ lệ chiến thắng của AI Solver trên các mức độ khó phổ biến.

Nhận xét: Tỷ lệ thắng cực kỳ xuất sắc ở bảng nhỏ. Ở bảng lớn (99 mìn), tỷ lệ giảm do mật độ mìn quá đặc, ép AI phải nhảy sang Tầng 3 (đoán 50/50) nhiều lần hơn.

Demo Trực tiếp (Live Demonstration)



Hạn chế & Hướng phát triển (Limitations & Future Works)

- Hạn chế:
 - Xác suất Tầng 3 chỉ tính cục bộ, chưa gom tính toàn cục.
 - Động cơ AI chạy cùng luồng (Main thread) với giao diện Tkinter -> bảng quá lớn (999 mìn) có thể gây tụt frame nhẹ.
- Hướng phát triển:
 - Nâng cấp Xác suất Toàn cục (Global Probabilities).
 - Thiết kế Đa luồng (Multi-threading) tách biệt AI và UI.
 - Dùng các Engine mạnh hơn như Pygame để thêm hiệu ứng (vụ nổ, âm thanh).